

Python fördjupning - DuckDB

Carina Karlsson - DS25

Syfte

Jag valde DuckDB som ämne då jag vill friska upp minnet gällande SQL, samt att jag var nyfiken på att använda DuckDB för att läsa och undersöka data, och även hur det skiljer sig från Pandas - vilka fördelar och nackdelar det har.

Området och dess relevans

För att testa detta skapade jag ett mindre projekt som läser in en CSV-fil och analyserar data, både med Pandas och DuckDB. Relevansen för en data scientist är att man ofta behöver läsa och analysera stora mängder data, då passar DuckDB in då det är snabbare än Pandas. Sen förekommer SQL ofta i rollen, vilket gör att det är bra att hålla den kunskapen färsk.

Viktiga begrepp

Några viktiga saker att veta gällande DuckDB: den är in-process, vilket betyder att den existerar inuti python-processen, den behöver ingen separat databasserver. Den behöver med andra ord inte skapa en anslutning för att göra analyser, utan man kan göra förfrågningar direkt genom SQL utan att först läsa in filen manuellt i minnet.

Man kan skapa en explicit anslutning genom att använda sig av `connect()`. Dock gör detta inte att det arbete du gör sparas, utan så fort man stänger ned programmet så försvinner datan. För att skapa en persistent anslutning så behöver man ha med ett argument till `connect()`, detta för att skapa en databasfil, som exempel: `import duckdb con = duckdb.connect(database='my_local_data.duckdb')`

DuckDB använder sig av kolumnlagring, vilket betyder att den bara läser in de kolumner den behöver för frågan, och läser inte resten.

DuckDB gör analyser av CSV, Parquet och JSON-filer med hjälp av SQL.

Genomförande

Jag visste att jag ville göra ett litet analysprojekt, så jag gick igenom tidigare CSV-filer tills jag hittade ett med passande data att undersöka och som var lite större i storlek. Till att börja med skapade jag en .py-fil för Pandas förfrågningar, och en annan för DuckDB, dessa har funktioner som analyserar datan och det är sen dessa som används i benchmark. Jag installerade duckdb, pandas och matplotlib. Genom att undersöka CSV-filen och dess kolumner, så valde jag några som passade att undersökas, som till exempel inkomst per månad, eller inkomst per kategori.

Sedan skapade jag benchmark, där jag fick ta hjälp av AI för hur man får fram hur lång tid en förfrågan tar och hur man visar upp det. Benchmarken använder `time.perf_counter()`

och varje test körs flera gånger. Medianvärdet används för att minska påverkan från enskilda ovanligt snabba eller långsamma körningar. Resultatet sparas i en CSV-fil och kan även visualiseras som ett stapeldiagram med Matplotlib.

Jag skapade även en testsida så att säga för några SQL förfrågningar, detta för att få övat lite på SQL, vilket var en av anledningarna till mitt val av fördjupning.

Sen skapade jag även en visuell .py-fil, för att se hur man skapar ett diagram utan att använda sig av Pandas. Här gör jag en förfrågan genom SQL och sparar resultatet till en DataFrame som jag sedan använder för att skapa ett diagram med matplotlib.

När jag körde benchmark upptäckte jag att DuckDB var långsammare än Pandas, vilket inte borde vara så, så jag undersökte detta och kom fram till att när Pandas körs så sparar den filen i minnet och använder den sedan för varje fråga, medans hur jag hade DuckDB upplagt så läste den filen varje gång i samband med frågan. På så sätt så bestod viss del av tiden av att läsa in CSV-filen. För att komma runt detta så använde jag mig av en explicit anslutning, connect(). På så sätt återanvänds anslutningen för analyserna, och skapar inte en ny anslutning varje gång.

En annan sak jag fick ändra var storleken på CSV-filen. Den första jag använde var med ca 2500 rader, inte riktigt stor nog för att visa vad DuckDB går för och visa skillnaden mellan den och Pandas. Jag fick skapa en större fil med samma kolumner, denna gång på 100 000 rader. Nu var skillnaden mycket tydligare i hastighet.

Resultat

Samma fyra analyser (`revenue_by_category`, `category_summary`, `monthly_revenue`) kördes i både Pandas och DuckDB, och tiden mättes som medianvärdet av 10 körningar per fråga:

```
def measure(function, number_of_runs=NUMBER_OF_RUNS):
    times = []

    for _ in range(number_of_runs):
        start = time.perf_counter()
        function()

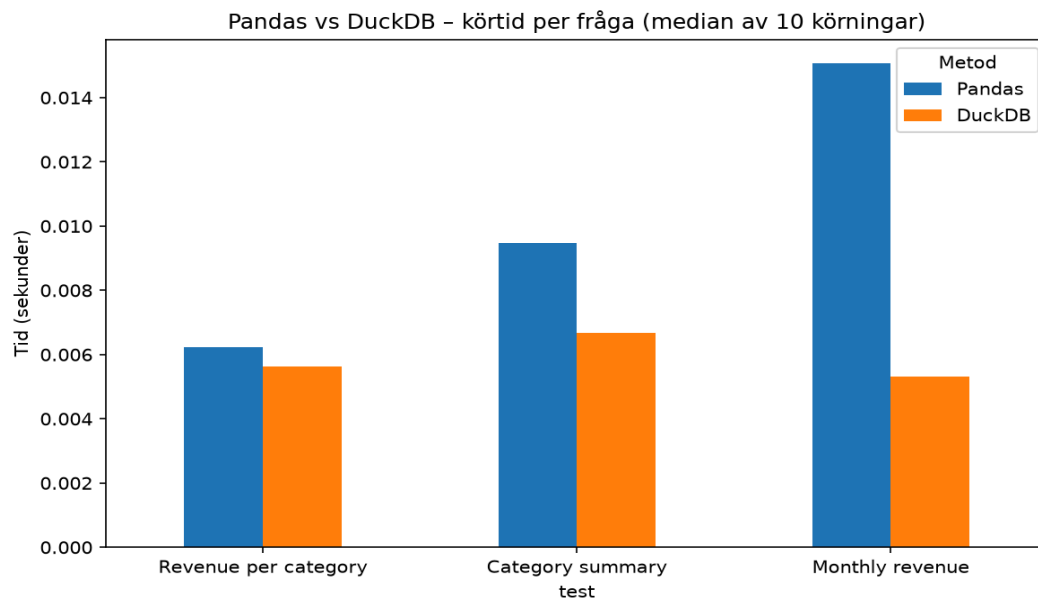
        end = time.perf_counter()
        times.append(end - start)

    return statistics.median(times)
```

Resultatet visar att DuckDB är snabbare än Pandas när det gäller att analysera data, dock gäller det när filen är stor; för mindre filer är skillnaden mellan dem försumbar.

	test	pandas_seconds	duckdb_seconds
0	Revenue per category	0.006232	0.005636
1	Category summary	0.009467	0.006682
2	Monthly revenue	0.015063	0.005314

Resultatet av det visualiseras sedan som ett stapeldiagram:



Detta visar att samma analys kan genomföras med Pandas och DuckDB och att verktygen erbjuder olika sätt att arbeta med data

För den visuella delen, där jag gör en analys av intäkt per kategori och gör ett diagram av det, genom att använda DuckDB för analysen och matplotlib för diagrammet:

```
DATA_FILE = "data/ecommerce_sales_100k.csv"
```

```
def create_revenue_chart():
    con = create_connection()
    result = con.execute(f"""
        SELECT
            category,
            SUM(revenue) AS revenue
        FROM '{DATA_FILE}'
        GROUP BY category
        ORDER BY revenue DESC
    """).df()

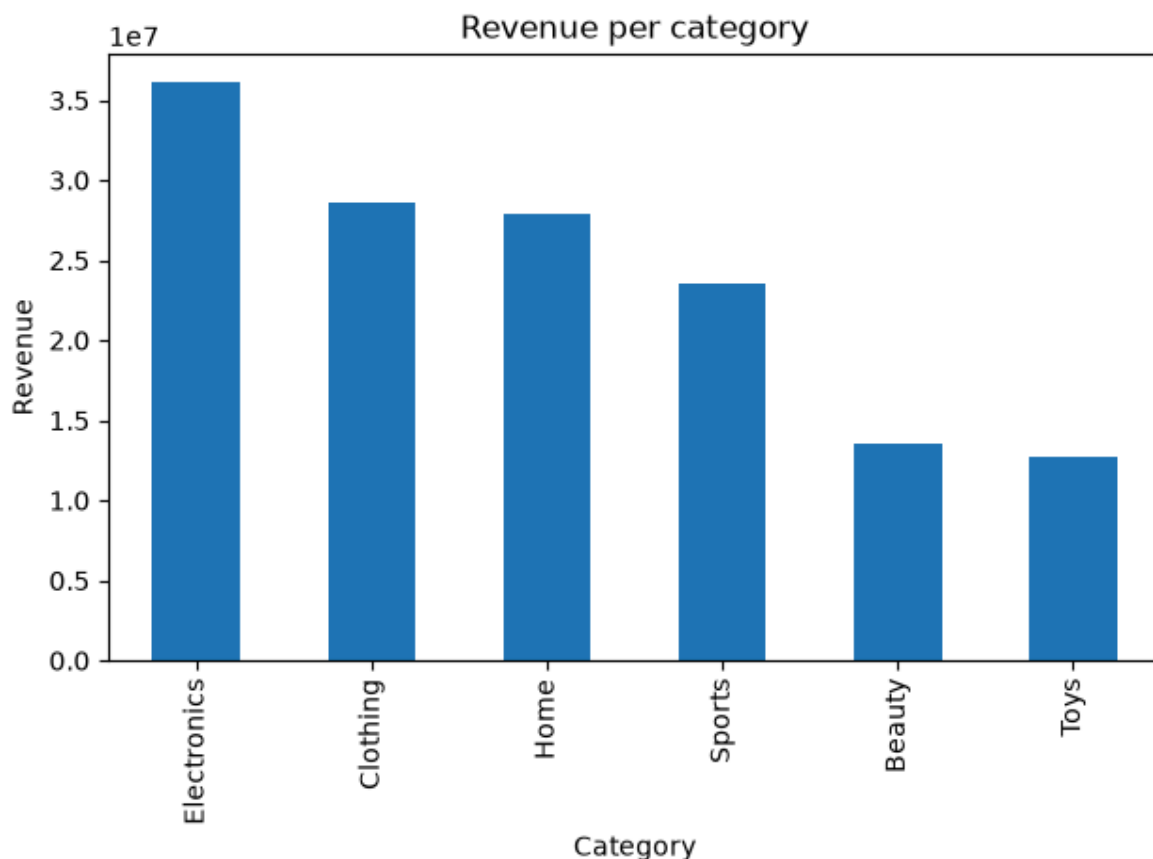
    result.plot(
        x="category",
        y="revenue",
        kind="bar",
        legend=False
    )

    plt.title("Revenue per category")
    plt.xlabel("Category")
    plt.ylabel("Revenue")
    plt.tight_layout()

    plt.show()
```

```
if __name__ == "__main__":
    create_revenue_chart()
```

Med resultatet i form av ett diagram:



Begränsningar och möjliga förbättringar

DuckDB är inte en ersättning till Pandas, utan ett komplement. Projektet är en förenklad jämförelse mellan Pandas och DuckDB och täcker därför inte alla användningsområden för de två. Testerna är dessutom begränsade till ett antal relativt enkla aggregeringar och tidsbaserade analyser. Mer avancerade analyser och större datamängder hade kunnat ge större skillnader mellan de två. Jag testar bara mot CSV-filer, inte mot till exempel Parquet eller att arbeta mot URL:er.

Den tydligaste begränsningen är att DuckDBs prestanda fördel inte är garanterad, utan beror på datamängdens storlek. En annan begränsning är att benchmarken bara mäter körtid för själva frågekörningen — inte utvecklingstid, läsbarhet eller hur lätt koden är att underhålla, vilket också är viktigt att ta hänsyn till i arbetslivet.

Möjliga förbättringar kan vara att testa flera datamängder och plotta hur körtiden växer för respektive verktyg. Även testa flera filformat, särskilt Parquet, eftersom DuckDB är särskilt optimerat för kolumnära format och skillnaden mot Pandas sannolikt hade varit ännu tydligare där.

Koppling till yrkesrollen

DuckDB kan vara användbart för en Data Scientist eller dataanalytiker som behöver bearbeta och analysera större mängder data. En Data Scientist arbetar ofta med data som

behöver filtreras, aggregeras och förberedas innan den kan användas för exempelvis statistisk analys eller maskininlärning. DuckDB kan användas för dessa steg med SQL direkt från Python. En styrka är möjligheten att kombinera DuckDB med Pandas. DuckDB kan exempelvis användas för SQL-baserad filtrering och aggregering, varefter resultatet konverteras till en Pandas DataFrame för visualisering, statistisk analys eller modellering. På så sätt kan SQL och Python användas tillsammans i samma arbetsflöde.

Källor

- DuckDB – officiell Python-dokumentation: <https://duckdb.org/docs/api/python/overview>
- DuckDB – CSV-läsning: <https://duckdb.org/docs/data/csv/overview>
- DuckDB – SQL-introduktion: <https://duckdb.org/docs/sql/introduction>
- <https://motherduck.com/learn/duckdb-python-quickstart-part1/>
- <https://betterstack.com/community/guides/scaling-python/duckdb-python/>
- <https://www.dench.com/blog/duckdb-for-data-science>
- <https://motherduck.com/blog/duckdb-tutorial-for-beginners/>

6. Självreflektion

1. Vad lärde du dig som du inte kunde innan?

Ett nytt verktyg, att använda SQL direkt i Python utan en databas som docker. Hur man gör en benchmark.

2. Vad var svårast att förstå eller genomföra?

Att skapa och förstå hur man gör en benchmark med att testa tid mm. Att sätta sig in i SQL igen, hade glömt en hel del.

3. Vilket tekniskt val är du mest nöjd med och varför?

Att jag använde connect() för själva jämförelserna, det gjorde skillnad när det gäller resultatet för DuckDB vs Pandas. Innan dess tog det längre tid då den var tvungen att läsa in CSV-filen varje gång.

4. Vad hade du gjort annorlunda om du började om?

Jag hade nog testat ännu större CSV-filer för att verkligen få en tydlig skillnad på jämförelserna. Och jag hade nog försökt testa att använda Parquet, jag är nyfiken på

hur det funkar ihop med DuckDB. Och hade gärna velat testa hur mycket minne som går åt för jämförelserna i benchmark och inte bara tid.

5. Vad skulle vara ett naturligt nästa steg om du fortsatte arbetet?

Större CSV-fil, även testa andra sorters filer såsom Parquet. Bygga ut med validering, tex.

6. **Vilket betyg tycker du själv att arbetet motsvarar – G eller VG?**

G.

7. **Motivera din bedömning genom att koppla till kraven för G och VG nedan.**

Jag har valt ett relevant område, och har avgränsat det tydligt, även förklarat varför det är relevant.

Jag har satt mig in i ämnet och byggt ett mindre projekt som fungerar. Har kunnat förklara de viktigaste begreppen och hur projektet fungerar.

Koden tycker jag är välstrukturerad och tydlig, har dokumentation hur projektet körs samt angett relevanta beroenden, data och källor.

Jag har skapat en skriftlig rapport där jag beskrivit syftet, genomförandet, resultaten och begränsningarna. Den innehåller även koppling till Data science yrkesrollen.

Slutligen har jag genomfört den muntliga presentationen.